

INTERVIEW QUESTIONS TO LAND AN AI ENGINEER JOB

AI System Design Foundations

*System design interviews test your decisions,
not your diagram vocabulary.*

Q01

A team built an AI system without spending adequate time on requirement clarification. Six months later, what are the five most likely consequences?

A

INTERVIEW-READY ANSWER

CONSEQUENCE 1: THE EVALUATION METRIC MEASURES THE WRONG THING

The team built a system optimized for a metric that doesn't correlate with actual user value. Maybe they optimized for retrieval recall but users care about response conciseness. The system scores well in internal evals but users don't find it helpful.



This is discovered 6 months in when user engagement metrics are flat despite positive internal benchmarks.

CONSEQUENCE 2: THE QUALITY BAR IS WRONG FOR THE USE CASE

Either the system is over-engineered (expensive and slow for a use case that didn't need it) or under-engineered (95% accuracy was achieved but the use case actually needed 99% or it causes errors users can't tolerate). Matching quality to stakes is a requirements decision that can't be made without knowing the use case.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q01

A team built an AI system without spending adequate time on requirement clarification. Six months later, what are the five most likely consequences?

A

INTERVIEW-READY ANSWER · CONTINUED



Consequence 3: The system doesn't handle the real input distribution

The prototype was built on clean, representative examples. Production input is messier, more diverse, and includes edge cases the team never considered. The system handles 70% of inputs well and completely fails on the other 30%.

POINT

WHAT IT MEANS

Consequence 4: The wrong data was collected or retained

Data requirements weren't defined upfront. The logging system captured the wrong things, didn't capture enough, or captured data in a format that isn't useful for evaluation or fine-tuning. Retroactively fixing data collection means either waiting for new data to accumulate or having no way to improve the system systematically.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q01

A team built an AI system without spending adequate time on requirement clarification. Six months later, what are the five most likely consequences?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

Consequence 5:
Major rework when
the real
requirements
emerge

The hardest to fix. The architecture was built for the assumed use case. When the real use case is clarified (usually after the first stakeholder demo or the first user feedback cycle), the architecture needs significant changes. This isn't a refactor, it's a rebuild.



Components that were core to the original design are now obstacles.



The hardest consequence to fix is usually the evaluation metric, because every optimization decision made for six months was made against the wrong metric.

Q02

You've deployed an AI system that worked in staging but fails in production within 48 hours. What are the five most common root causes and how would you have designed to prevent each?

A

INTERVIEW-READY ANSWER



Root cause 1: Distribution shift between staging and production data

Staging used a sample of clean, representative data. Production receives the full, messy distribution including edge cases, unusual languages, adversarial inputs, and data formats the team didn't anticipate.

POINT

WHAT IT MEANS

Prevention

shadow mode testing before hard launch. Route production traffic to the staging system in shadow mode (responses are discarded, not served) and monitor how the system handles real traffic before it handles it for real.

Root cause 2

Third-party API behavior differs at scale

The external model API or vector store behaved consistently in staging at low load. At production load, you hit rate limits, experience different latency percentiles, or encounter API behaviors that only manifest at scale.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q02

You've deployed an AI system that worked in staging but fails in production within 48 hours. What are the five most common root causes and how would you have designed to prevent each?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Prevention	load test at 2x expected production volume in a staging environment connected to production APIs (on a test endpoint or with test data). Monitor p99 latency, not just mean latency.
Root cause 3	Context window overflow
In staging, test conversations were short. In production, users have longer conversations, submit longer documents, or trigger more retrieval results than anticipated. Prompts start exceeding the context window, causing truncation or errors.	
Prevention	define maximum input sizes and enforce them in preprocessing. Test with the maximum allowed inputs, not just typical ones. Monitor prompt length distribution in production.
Root cause 4	Cold start and caching behavior

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q02

You've deployed an AI system that worked in staging but fails in production within 48 hours. What are the five most common root causes and how would you have designed to prevent each?

A

INTERVIEW-READY ANSWER · CONTINUED

The staging environment was warm from testing. Production started cold. The first wave of production traffic hit an unwarmed cache, uncompiled models, and cold infrastructure, causing a slow start that compounded into a failure cascade.

POINT

WHAT IT MEANS

Prevention

warm the production environment before traffic arrives. Load the model into memory, populate the cache with common queries, and verify component health before directing traffic.

Root cause 5

Missing monitoring and alerting



The system failed silently for hours before anyone noticed. There was no alerting on error rate, no latency monitoring, no output quality checks.

Prevention: define success metrics and alert thresholds before deployment. The monitoring should be deployed before the system, not after. A system without monitoring is not production-ready.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q03

You have 15 minutes to present your AI system design to a CTO who is not an ML expert. How do you structure it?

A

INTERVIEW-READY ANSWER

A CTO who is not an ML expert needs to understand three things: what the system does, why the key architectural decisions were made, and what could go wrong. Everything else is detail they can get from the engineers.

MINUTES 1-2: WHAT THE SYSTEM DOES, IN PLAIN LANGUAGE

"This system lets a customer service agent ask a question in plain English and get an answer drawn from our entire knowledge base, sourced and cited, within 2 seconds." No mention of transformers, embeddings, or RAG yet.



Minutes 3-5: The key components, with analogies

USE ANALOGIES THE CTO WILL RECOGNIZE



"There's a search layer that works like Google but for our internal documents, except instead of matching keywords it matches meaning."

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q03

You have 15 minutes to present your AI system design to a CTO who is not an ML expert. How do you structure it?

A

INTERVIEW-READY ANSWER · CONTINUED



"There's an AI layer that reads the search results and writes a coherent answer, like a junior analyst who reads the relevant documents and summarizes them."



"There's a validation layer that checks whether the answer looks reasonable before it's shown to the user."



Minutes 6–9: Why we made key decisions

FOCUS ON TWO OR THREE DECISIONS THAT WERE GENUINELY NON-OBVIOUS



"We chose to use retrieval rather than training a custom model because our knowledge base changes weekly. Retraining would take months and be expensive."



"We built a human review step for high-confidence queries because we have strict accuracy requirements for regulatory compliance."

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q03

You have 15 minutes to present your AI system design to a CTO who is not an ML expert. How do you structure it?

A

INTERVIEW-READY ANSWER · CONTINUED



Frame decisions in terms of business impact, not technical preference.



Minutes 10–12: What could go wrong and how we've handled it

CTOS ARE RISK-ORIENTED. ADDRESS THE TOP TWO OR THREE RISKS PROACTIVELY



"If the AI gives a wrong answer, users see a disclaimer and a link to the source document to verify."



"If the API we depend on goes down, we have a fallback to a simpler search experience."

MINUTES 13-15: QUESTIONS

Save time for questions. CTOs often have specific concerns they want addressed. The best 15-minute presentation ends with the CTO asking the exact question you were hoping for.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q04

Your AI system is suddenly expected to handle 50x the traffic after a viral marketing campaign. Which components fail first and how do you redesign to handle the surge?

A

INTERVIEW-READY ANSWER

50x traffic is a stress test that reveals every assumption baked into the original architecture. Let me walk through what fails in order.

WHAT FAILS FIRST

POINT	WHAT IT MEANS
LLM inference capacity	This is almost always the first failure point. If you're using an external API, you'll hit rate limits immediately. If you're self-hosted, GPU utilization goes to 100% and latency spikes. LLM inference doesn't scale horizontally as easily as stateless services.
Vector store query load	If you're using a vector database, 50x queries means 50x index queries. Many vector stores have limits on concurrent queries, and some configurations (single-instance deployments) will become a bottleneck quickly.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q04

Your AI system is suddenly expected to handle 50x the traffic after a viral marketing campaign. Which components fail first and how do you redesign to handle the surge?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Embedding generation	Less often the bottleneck but still a concern at 50x. Embedding models are fast but not free.
Application servers and API gateways	These are usually the easiest to scale horizontally but they'll still need attention.

HOW TO REDESIGN FOR SURGE HANDLING

POINT	WHAT IT MEANS
For inference	Move to a provider with auto-scaling (many managed API providers handle this). Implement aggressive caching for common queries. Consider routing simple queries to a smaller, faster model and reserving the expensive model for complex queries.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q04

Your AI system is suddenly expected to handle 50x the traffic after a viral marketing campaign. Which components fail first and how do you redesign to handle the surge?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
For retrieval	Add a caching layer in front of the vector store. Common query patterns often retrieve similar documents. A cache with a short TTL reduces vector store load significantly.
Circuit breakers	Implement circuit breakers that degrade gracefully under load: return cached results, simplified responses, or a friendly "we're experiencing high demand" message rather than timing out or returning errors.
Request queuing	Instead of failing requests immediately when capacity is exceeded, queue them with a short TTL. Users can tolerate 10-second waits better than errors.
Horizontal scaling with auto-scaling rules	Every stateless component (preprocessing, postprocessing, API server) should have auto-scaling configured with appropriate rules.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q05

Halfway through your design session, you realize a critical piece of information is missing – you don't know whether the data is real-time or batch. How do you continue?

A

INTERVIEW-READY ANSWER

Missing information halfway through a design session is normal. The mistake is either to stop completely or to silently assume. The right move is to design for both while flagging the fork.

EXPLICITLY NAME THE UNCERTAINTY

"I just realized we haven't established whether this needs to be real-time or batch, and that's actually a fork in the architecture. Let me design both paths and we can converge once we know."



This is a mark of maturity, not a sign of weakness. It demonstrates you understand the implications.

IDENTIFY WHAT'S SHARED BETWEEN BOTH PATHS

DESPITE THE REAL-TIME VS BATCH DISTINCTION, MANY COMPONENTS ARE SHARED



Document ingestion and preprocessing



Embedding generation and indexing

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q05

Halfway through your design session, you realize a critical piece of information is missing – you don't know whether the data is real-time or batch. How do you continue?

A

INTERVIEW-READY ANSWER · CONTINUED



Data storage

OUTPUT FORMATTING

Design these shared components first. They're valid regardless of which path you take.

DESIGN BOTH DIVERGENT PATHS

POINT	WHAT IT MEANS
Real-time path	user makes a request, retrieval happens synchronously, generation happens synchronously, response is returned within SLA. Components need to be low-latency and horizontally scalable.
Batch path	jobs are triggered on a schedule or by events, retrieval and generation can take more time, results are written to storage for later consumption. Components can be optimized for throughput over latency.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q05

Halfway through your design session, you realize a critical piece of information is missing – you don't know whether the data is real-time or batch. How do you continue?

A

INTERVIEW-READY ANSWER · CONTINUED

DOCUMENT THE IMPLICATIONS OF EACH CHOICE

POINT	WHAT IT MEANS
For real-time	latency requirements constrain model size and retrieval depth. Infrastructure needs to scale for peak concurrent load.
For batch	throughput matters more than latency. Can use larger models, deeper retrieval, more complex postprocessing.

IN AN INTERVIEW CONTEXT

If this is an interview question, make an assumption and state it clearly: "I'm going to assume real-time since that's the harder constraint, but I'll note the batch variant. Correct me if you had something else in mind." Then proceed.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q06

You're designing a pipeline that depends on a third-party enrichment API called mid-pipeline. What are the hidden risks and how do you design around them?

A

INTERVIEW-READY ANSWER

A third-party API mid-pipeline is a dependency that you don't control sitting in the critical path of your system. That's a reliability risk, a latency risk, a cost risk, and a data risk simultaneously.

HIDDEN RISK 1: CASCADING FAILURE

If the enrichment API goes down, your entire pipeline stalls. This is the most immediate risk. If your pipeline is synchronous and the API times out, users see failures.

POINT

WHAT IT MEANS

Design
around it

implement a circuit breaker that detects when the API is failing and switches to a fallback mode (cached enrichment data, degraded but functional pipeline without enrichment, or a queued retry mechanism).

Hidden risk
2

Latency coupling



The third-party API's performance becomes your pipeline's performance. If they have a slow day, your p99 latency spikes.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q06

You're designing a pipeline that depends on a third-party enrichment API called mid-pipeline. What are the hidden risks and how do you design around them?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Design around it	set aggressive timeouts on the API call. If enrichment is non-critical, make it async and proceed without it if it doesn't return within the timeout. Cache enrichment results aggressively since they're often stable over short periods.
Hidden risk 3	Cost unpredictability



Third-party APIs often have per-call pricing. At scale, enrichment API costs can exceed your primary model costs if you're not careful.

POINT	WHAT IT MEANS
Design around it	cache enrichment results. Before calling the API, check whether you've already enriched this entity recently. Set budget alerts and circuit breakers on per-day spend.
Hidden risk 4	Schema changes

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q06

You're designing a pipeline that depends on a third-party enrichment API called mid-pipeline. What are the hidden risks and how do you design around them?

A

INTERVIEW-READY ANSWER · CONTINUED

- ⚡ The third-party API changes its response schema without notice. Your downstream pipeline breaks silently because it parses a field that no longer exists.

POINT

WHAT IT MEANS

Design
around it

validate the API response schema on every call. Log schema violations. Build tolerant parsers that handle optional fields gracefully and alert on unexpected changes.

Hidden risk
5

Data privacy implications



You're sending user data to a third-party service. Does their data policy permit this? Are you covered under your privacy policy?



Design around it: audit what data you're sending to the enrichment API. Anonymize or hash identifying information before sending if the enrichment doesn't require it.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q07

Your stakeholders want real-time responses, zero hallucinations, full audit logs, and sub-\$0.001 per query cost. These requirements conflict. How do you facilitate a trade-off conversation?

A

INTERVIEW-READY ANSWER

These four requirements can't all be maximized simultaneously. My job isn't to tell stakeholders this is impossible. It's to help them understand the trade-offs so they can make an informed decision.

FIRST, SHOW THE CONFLICT EXPLICITLY

PRESENT A SIMPLE TRADE-OFF MATRIX SHOWING WHICH COMBINATIONS ARE REALISTIC

Real-time response + zero hallucinations requires either a very fast model with extensive retrieval verification (expensive) or a smaller model with lower accuracy (more hallucinations). You can't have both at sub-\$0.001.



Full audit logs + real-time response adds latency for log writing. It also increases storage cost significantly at scale.

↓ **CHECK THE CAPTION FOR THE FULL INTERVIEW KIT**

Q07

Your stakeholders want real-time responses, zero hallucinations, full audit logs, and sub-\$0.001 per query cost. These requirements conflict. How do you facilitate a trade-off conversation?

A

INTERVIEW-READY ANSWER · CONTINUED



Sub-\$0.001 per query is achievable with small models and minimal retrieval, but those configurations have higher hallucination rates.



Then ask: what is the relative priority?

Ask each stakeholder to rank the four requirements and explain what happens if each one is violated. Often, the conversation reveals that "zero hallucinations" really means "low hallucination rate for important claims" or "real-time" means "under 3 seconds" not under 500ms.

PROPOSE A TIERED DESIGN

DIFFERENT QUERY TYPES GET DIFFERENT HANDLING

POINT	WHAT IT MEANS
Simple, high-confidence queries	fast small model, low cost, acceptable hallucination rate for low-stakes information

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q07

Your stakeholders want real-time responses, zero hallucinations, full audit logs, and sub-\$0.001 per query cost. These requirements conflict. How do you facilitate a trade-off conversation?

A INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

Complex, high-stakes queries

full RAG, verification step, complete audit log, higher cost



This architecture lets you meet the budget constraint for most queries while meeting the quality constraint for the ones that matter.

GET EXPLICIT SIGN-OFF ON THE TRADE-OFFS CHOSEN

Write down what was agreed: "We are accepting X% hallucination rate on low-stakes queries in exchange for sub-\$0.001 cost at that tier." This creates accountability and prevents requirements from expanding after the design is locked.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q08

Your system has four components: embedding generation, vector retrieval, LLM inference, and response formatting. Under load, one of them will bottleneck first. How do you identify which one before it happens in production?

A

INTERVIEW-READY ANSWER

The component that bottlenecks first is the one with the lowest throughput relative to its demand. You find it through profiling before production and monitoring in production.



Step 1: Measure the latency distribution of each component independently

Instrument each component separately with timing code. Run synthetic load tests that isolate each stage. You're looking for mean latency, p95, p99, and behavior under increasing concurrent load. The component whose p99 degrades earliest as load increases is your likely bottleneck.

TYPICAL ROUGH BENCHMARKS TO SET EXPECTATIONS

POINT	WHAT IT MEANS
Embedding generation	10–50ms per request on GPU, faster with batching, much slower on CPU

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q08

Your system has four components: embedding generation, vector retrieval, LLM inference, and response formatting. Under load, one of them will bottleneck first. How do you identify which one before it happens in production?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Vector retrieval	20–100ms depending on index size, hardware, and number of results
LLM inference	100ms–2 seconds depending on model size, hardware, and output length
Response formatting	typically sub-10ms unless it involves complex post-processing



In most systems, LLM inference is the bottleneck by a wide margin. It's compute-intensive, doesn't parallelize easily per-request, and doesn't scale linearly with more instances.

STEP 2: MODEL THE SYSTEM UNDER LOAD BEFORE TESTING

Before running load tests, do a back-of-envelope calculation: if you're targeting 100 RPS and LLM inference takes 500ms, you need 50 concurrent inference slots just to keep up. If your GPU can handle 10 concurrent requests, you're already 5x under capacity.

↓ [CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q08

Your system has four components: embedding generation, vector retrieval, LLM inference, and response formatting. Under load, one of them will bottleneck first. How do you identify which one before it happens in production?

A

INTERVIEW-READY ANSWER · CONTINUED



This math often reveals the bottleneck before you've written a line of load test code.

POINT

WHAT IT MEANS

Step 3: Load test with realistic traffic patterns

Run a load test with realistic traffic: gradual ramp-up, sustained peak, and spike scenarios. Monitor each component's queue depth, not just latency. A component whose queue grows under load is the bottleneck, even if its per-request latency looks acceptable.

Step 4: Design your monitoring before deployment

Add metrics for each component: request rate, latency percentiles, queue depth, error rate, and resource utilization. Set alerts before deployment. You want to know which component is under stress in production before users start complaining.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q09

Your AI system produces slightly different outputs for the same input on every run. How does this non-determinism affect the downstream components that depend on it?

A

INTERVIEW-READY ANSWER

Non-determinism isn't a bug you can fix. It's a property of the system you have to design around. The question is what assumptions your downstream components are making that this property violates.

COMPONENTS THAT BREAK WITHOUT CAREFUL HANDLING

Caching layers: Traditional caches assume the same input always produces the same output. If you cache LLM responses, you're making a bet that the cached response is acceptable every time that cache key is hit. Sometimes that's fine.



Sometimes the response quality variance matters and you shouldn't cache at all, or you should cache with shorter TTLs.

Q09

Your AI system produces slightly different outputs for the same input on every run. How does this non-determinism affect the downstream components that depend on it?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Deterministic testing	Test suites that assert exact output values will fail constantly in AI systems. You need to shift to semantic assertions ("the response mentions the customer's name") or use a separate evaluator model to check whether the output meets quality criteria, rather than checking for exact string matches.
Downstream systems that take action on output	If an AI system routes a customer to a specific support queue and the routing decision varies run-to-run, the downstream queue management system needs to handle inconsistency. Design these integrations with soft handoffs rather than hard dependencies.
Audit and logging	When you log AI responses for compliance or debugging, the log for the same input at two different times will show different content. This is expected but confusing if your audit trail assumes deterministic system behavior.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q09

Your AI system produces slightly different outputs for the same input on every run. How does this non-determinism affect the downstream components that depend on it?

A

INTERVIEW-READY ANSWER · CONTINUED

PRACTICAL DESIGN RESPONSES



Set temperature to 0 for any system path that requires determinism (safety checks, structured output parsing, classification decisions)



Log the full context alongside every response so you can understand why a specific output was produced



Design downstream systems to be tolerant of output variance within an acceptable quality range



Use output schema validation to catch structural failures even when content varies

Q10

Your system must respond in under 150ms end-to-end, including model inference. Walk through how this single constraint shapes every architectural decision.

A

INTERVIEW-READY ANSWER

150ms end-to-end is a very tight budget for any AI system. This single constraint eliminates a large number of common architectural choices before you even start designing.

BUDGET ALLOCATION FIRST

Break the 150ms into a rough budget per stage. A realistic allocation might be: 10ms for input preprocessing, 20ms for retrieval (if any), 100ms for model inference, 15ms for output postprocessing and formatting, and 5ms for network overhead. If any stage reliably exceeds its budget, the whole system fails the SLA.

WHAT THIS MEANS FOR MODEL CHOICE

Large models (70B+ parameters) are immediately off the table unless you have serious GPU infrastructure and high batch throughput. You're looking at smaller, faster models (7B–13B range) served on high-performance inference infrastructure, or API providers that offer dedicated throughput with latency SLAs. GPT-4 on a shared endpoint is unlikely to reliably hit 100ms inference.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q10

Your system must respond in under 150ms end-to-end, including model inference. Walk through how this single constraint shapes every architectural decision.

A

INTERVIEW-READY ANSWER · CONTINUED

⚡ A smaller specialized model might.

WHAT THIS MEANS FOR RETRIEVAL

Traditional RAG with full vector search adds 50–200ms in typical configurations. At 150ms total budget, you have almost no room for vector search. Options: an in-memory cache for common queries (near-zero latency), pre-computed retrieval results for predictable query patterns, or accepting that retrieval is impossible at this latency target and designing without it.

WHAT THIS MEANS FOR THE SYSTEM PROMPT

🗄️ Long system prompts increase inference time because they increase token processing. Keep the system prompt minimal. Pre-tokenize and cache it at the inference layer.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q10

Your system must respond in under 150ms end-to-end, including model inference. Walk through how this single constraint shapes every architectural decision.

A

INTERVIEW-READY ANSWER · CONTINUED

WHAT THIS MEANS FOR OUTPUT PROCESSING

No synchronous output validation using a second model call. If you need output validation, it has to be lightweight (regex, schema validation) or async (validate after delivery and handle failures separately).

WHAT THIS MEANS FOR INFRASTRUCTURE

Co-locate inference and application servers to minimize network round-trips. Consider model quantization or distillation to reduce inference time. Implement aggressive caching at every layer for repeated or similar inputs.

THE HONEST ANSWER

150ms end-to-end is possible but requires very specific trade-offs: small fast models, no complex retrieval, minimal post-processing. If the PM wants this latency AND high quality AND full RAG AND output validation, something has to give. This constraint is the right conversation to have before writing code.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q11

You're building a document classification system. The business wants 99% accuracy. Engineering says that achieves it requires 5x infrastructure cost and 3x latency. How do you navigate this?

A

INTERVIEW-READY ANSWER

This is a stakeholder alignment problem disguised as a technical problem. The business wants a number without understanding what that number costs.

STEP 1: REFRAME THE CONVERSATION AROUND COST OF ERRORS

99% accuracy sounds like a simple requirement. But what it really means is: 1 in 100 documents is classified incorrectly. The question I'd ask the business is: what happens when a document is misclassified? Is it routed to the wrong team and sits in a queue for a day?



Or does it trigger a financial transaction or a legal action? The cost of misclassification determines whether 99% is worth 5x infrastructure.



Step 2: Present the accuracy/cost curve, not just two endpoints

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q11

You're building a document classification system. The business wants 99% accuracy. Engineering says that achieves it requires 5x infrastructure cost and 3x latency. How do you navigate this?

A

INTERVIEW-READY ANSWER · CONTINUED

RATHER THAN FRAMING THIS AS "95% VS 99%", PRESENT THE FULL CURVE



90% accuracy: current cost, current latency



95% accuracy: 2x cost, 1.5x latency



97% accuracy: 3x cost, 2x latency

POINT

WHAT IT MEANS

99% accuracy: 5x cost, 3x latency

When stakeholders see the full curve, they often choose a middle point they didn't know existed.

Step 3: Propose a hybrid architecture

99% accuracy on all documents isn't necessary if you can triage. Design a system where:

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q11

You're building a document classification system. The business wants 99% accuracy. Engineering says that achieves it requires 5x infrastructure cost and 3x latency. How do you navigate this?

A INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

The model classifies with a confidence score

High-confidence classifications (say, 90%+ of volume) are processed automatically at lower cost and latency



Low-confidence classifications (10% of volume) are routed to human review or the more expensive model



This achieves near-99% effective accuracy on the full corpus at a fraction of the cost, because you're only paying for expensive processing on the hard cases.

STEP 4: SET A MEASUREMENT PLAN

Agree on how accuracy will be measured: on what test set, with what frequency, with what human review as ground truth. "99% accuracy" needs a measurement methodology or it's unmeasurable.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q12

After a model upgrade, accuracy improves by 15% but p99 latency doubles. Your product serves both a real-time chat interface and a batch reporting pipeline. How do you decide what to prioritize?

A

INTERVIEW-READY ANSWER

This is a classic engineering trade-off where the right answer depends on who is waiting for the response.

THE REAL-TIME CHAT INTERFACE

Users are waiting for a response. If p99 latency doubles and that puts you above the threshold where users perceive a lag (generally above 1-2 seconds for conversational interfaces), you have a UX problem that will show up in engagement metrics before it shows up in accuracy metrics.



Users don't notice 15% accuracy improvement directly. They immediately notice slowness.

For the chat interface, I'd keep the old model until I can close the latency gap, or I'd investigate whether I can reduce latency through caching common responses, parallel processing, or streaming the output so users see tokens arriving rather than waiting for the full response.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q12

After a model upgrade, accuracy improves by 15% but p99 latency doubles. Your product serves both a real-time chat interface and a batch reporting pipeline. How do you decide what to prioritize?

A

INTERVIEW-READY ANSWER · CONTINUED

THE BATCH REPORTING PIPELINE

Latency barely matters here. No one is waiting synchronously for a batch job. If the job runs overnight and takes 20% longer due to higher latency per request, that's usually acceptable. The 15% accuracy improvement on the other hand directly affects the quality of the reports, which affects business decisions made from those reports.



For the batch pipeline, I'd upgrade immediately and accept the latency increase.

THE GENERAL PRINCIPLE

Route the two use cases to different model versions. This isn't unusual. Serving different model versions to different pipeline types lets you optimize each use case independently. The infrastructure complexity is worth it because the trade-offs are genuinely different.

I'd also immediately start profiling where the latency increase comes from in the new model. Is it the model itself, or is it the context processing, output parsing, or something else I can optimize?

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q13

Your system will receive inputs from thousands of enterprise clients with different languages, formats, encodings, and schemas. How do you design the ingestion layer?

A

INTERVIEW-READY ANSWER

Enterprise input diversity is a reliability problem disguised as a feature request. The ingestion layer is your only chance to normalize the chaos before it reaches the rest of your system.



Design principle: normalize to a canonical format as early as possible

The rest of your pipeline should never know that the input was originally a Korean-language Excel file with a custom date format. By the time data reaches the retrieval or generation layer, it should be in one consistent format.

THE INGESTION LAYER ARCHITECTURE

DETECTION STAGE



Detect file format (PDF, DOCX, CSV, JSON, HTML, plain text) regardless of file extension



Detect encoding (UTF-8, Latin-1, etc.) and normalize to UTF-8

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q13

Your system will receive inputs from thousands of enterprise clients with different languages, formats, encodings, and schemas. How do you design the ingestion layer?

A

INTERVIEW-READY ANSWER · CONTINUED

DETECT LANGUAGE USING A LIGHTWEIGHT CLASSIFIER

Detect schema type for structured data (what fields are present, what are their types)

EXTRACTION STAGE



Format-specific extractors: a PDF extractor, a DOCX extractor, a spreadsheet extractor, etc.



Each extractor produces the same output schema: text content, metadata, source reference



Use sandboxed parsers that can't be crashed by malformed input

NORMALIZATION STAGE



Normalize text encoding to UTF-8



Normalize whitespace, line endings, special characters

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q13

Your system will receive inputs from thousands of enterprise clients with different languages, formats, encodings, and schemas. How do you design the ingestion layer?

A

INTERVIEW-READY ANSWER · CONTINUED

NORMALIZE DATE FORMATS AND NUMERIC FORMATS WHERE DETECTABLE

Apply language-specific preprocessing where needed (CJK character handling, right-to-left text)

VALIDATION AND ERROR HANDLING STAGE



Validate that the extracted content meets minimum quality thresholds (not empty, not binary garbage from a failed parse)



Reject or quarantine inputs that fail validation with specific error reasons



Log every rejection with enough context to debug

WHAT YOU NEVER DO

Pass raw client input directly to the model or the retrieval system. Every input goes through the full normalization pipeline, no exceptions.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q14

You have a \$500/month infrastructure budget for an AI feature that serves 10,000 users. How do you architect to stay within budget while maintaining acceptable quality?

A

INTERVIEW-READY ANSWER

\$500/month for 10,000 users is \$0.05 per user per month. That's extremely tight but workable with the right architecture decisions.

START WITH USAGE MATH

Not all 10,000 users are active daily. If 20% are active on any given day, that's 2,000 daily active users. If each makes 5 queries per day, that's 10,000 queries per day or roughly 7 queries per minute on average.



At 7 queries per minute average with peaks perhaps 3–5x that, you need infrastructure that handles roughly 35 queries per minute at peak.

Q14

You have a \$500/month infrastructure budget for an AI feature that serves 10,000 users. How do you architect to stay within budget while maintaining acceptable quality?

A

INTERVIEW-READY ANSWER · CONTINUED

MODEL SELECTION IS THE BIGGEST COST LEVER



A single GPT-4 query can cost \$0.01–\$0.10 depending on token count. At 10,000 queries per day, even \$0.02 average cost is \$200/day, completely blowing the budget.

OPTIONS WITHIN BUDGET

Open-source models (Llama, Mistral) hosted on a small GPU instance: a single A10G GPU instance on a cloud provider costs around \$0.90–\$1.50/hour. Running it 24/7 is \$650–\$1,080/month, but you can use spot instances and scale to zero when traffic is low.



API providers with cheaper smaller models: GPT-3.5-turbo or Claude Haiku at \$0.001/1K tokens can keep costs under \$50/month if prompts are short and responses are concise.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q14

You have a \$500/month infrastructure budget for an AI feature that serves 10,000 users. How do you architect to stay within budget while maintaining acceptable quality?

A

INTERVIEW-READY ANSWER · CONTINUED

CACHING IS YOUR BEST FRIEND

Cache responses for common or repeated queries. Even a 20% cache hit rate significantly reduces inference cost. Use semantic caching (cache if the embedding similarity to a prior query exceeds a threshold) rather than exact string matching.

EFFICIENT PROMPT DESIGN



Keep system prompts short. Don't retrieve more context than necessary. Limit response length where possible. Token count is cost.

BATCH WHERE YOU CAN

Any queries that aren't real-time should be batched. Batch processing is cheaper on most providers.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q15

Your LLM sometimes returns malformed outputs, sometimes refuses to answer, and sometimes hallucinates. How do you design a system that handles all three failure modes gracefully?

A

INTERVIEW-READY ANSWER

These three failure modes are different in nature and require different handling strategies.

MALFORMED OUTPUTS



The model doesn't follow the output format you specified. Maybe you asked for JSON and got prose, or asked for a list and got a paragraph.

Handling strategy: Implement output parsing with strict schema validation. When validation fails, retry the request (once, maybe twice) with the format requirement made more explicit in the prompt. If retries fail, return a structured error to the caller that triggers fallback behavior (a simpler template response, or a human escalation).



For high-volume production systems, structured output APIs (where the provider enforces the schema at the inference level) eliminate most malformed output issues.

Q15

Your LLM sometimes returns malformed outputs, sometimes refuses to answer, and sometimes hallucinates. How do you design a system that handles all three failure modes gracefully?

A

INTERVIEW-READY ANSWER · CONTINUED

REFUSALS



The model declines to answer because it interprets the request as violating its safety guidelines, even though the request is legitimate.

POINT

WHAT IT MEANS

Handling strategy

First, analyze your refusal rate and categorize refusals. If legitimate requests are being refused at a significant rate, the problem is usually in the prompt framing. Rephrase the system prompt to provide context that makes the legitimate use case clear to the model.

At the system level

detect refusals (they often have detectable patterns in phrasing), route them to a fallback that either reformulates the request with different framing or escalates to a human.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q15 Your LLM sometimes returns malformed outputs, sometimes refuses to answer, and sometimes hallucinates. How do you design a system that handles all three failure modes gracefully?

A INTERVIEW-READY ANSWER · CONTINUED

HALLUCINATIONS

 The model generates plausible but factually incorrect information.

POINT	WHAT IT MEANS
Handling strategy	The best defense is architectural. Don't rely on the model's parametric knowledge for factual claims. Use RAG to provide the facts as retrieved context, and instruct the model to answer only from the provided context. When the context doesn't contain an answer, the model should say so rather than generating from memory.
At the output level	for factual claims that can be verified (dates, names, numbers), run a verification step against a trusted source before returning the response.

None of these failure modes can be eliminated entirely. Design for their occurrence, not against their possibility.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q16

You're designing a pipeline where user input flows through preprocessing, retrieval, generation, and postprocessing. How does explicitly mapping data flow at each stage change the quality of your design?

A

INTERVIEW-READY ANSWER

Explicitly mapping data flow is not a documentation exercise. It's a design tool that reveals problems before you build anything.

WHAT A DATA FLOW MAP FORCES YOU TO DEFINE

FOR EACH STAGE TRANSITION, YOU'RE FORCED TO ANSWER



What is the exact schema of the data entering this stage?



What transformation happens inside this stage?



What is the exact schema of the data leaving this stage?



What can go wrong in this stage and what happens to the data flow when it does?

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q16

You're designing a pipeline where user input flows through preprocessing, retrieval, generation, and postprocessing. How does explicitly mapping data flow at each stage change the quality of your design?

A

INTERVIEW-READY ANSWER · CONTINUED

Most system designs fail at the stage transitions, not within the stages themselves. The preprocessing stage might work fine. The retrieval stage might work fine. But if preprocessing outputs a raw string and retrieval expects a structured query object, you've found a contract violation before it becomes a runtime bug.

THE CONCRETE IMPROVEMENTS THIS CREATES

POINT	WHAT IT MEANS
Interface contracts are explicit	Each stage has a defined input and output schema. When a stage is changed or optimized, the contract constrains what can change without breaking downstream components.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q16

You're designing a pipeline where user input flows through preprocessing, retrieval, generation, and postprocessing. How does explicitly mapping data flow at each stage change the quality of your design?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Error handling becomes visible	When you map what happens to data flow on failure at each stage, you find gaps. If the retrieval stage returns zero results, does generation get an empty list, a null, or an error? Without explicit data flow mapping, this is usually discovered in production.
Bottlenecks are predictable	When you know the data shape at each stage, you can estimate compute requirements. A stage that takes a 512-dimensional embedding vector and a list of 20 retrieved chunks and produces a 4,000-token prompt is going to have inference implications. You can see this before building.
Testing becomes granular	With explicit data flow, you can build unit tests for each stage with well-defined inputs and expected outputs. Without it, you end up with only integration tests that are slow and hard to debug.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q17

You're given the goal: "improve user experience using AI." How do you break this down into a concrete, scoped system with measurable outcomes?

A

INTERVIEW-READY ANSWER

"Improve user experience using AI" is a direction, not a specification. My job is to decompose it until it becomes buildable and testable.

STEP 1: IDENTIFY THE FRICTION POINT

I'd start by finding the specific moment in the user journey where the experience breaks down. Is it that users can't find what they're looking for? Are they getting irrelevant results? Are they doing repetitive work that could be automated? You can't improve user experience in general.



You can improve a specific step in a specific workflow.

STEP 2: TRANSLATE THE UX GOAL INTO AN AI TASK TYPE

Once I know the friction point, I can classify what kind of AI problem this actually is:

POINT	WHAT IT MEANS
Users can't find relevant content	this is a retrieval or search problem

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q17

You're given the goal: "improve user experience using AI." How do you break this down into a concrete, scoped system with measurable outcomes?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

Users are getting one-size-fits-all responses

this is a personalization problem

Users are doing repetitive data entry

this is an extraction or automation problem

Users can't get quick answers

this is a question-answering problem



The task type determines the system architecture entirely.

STEP 3: DEFINE MEASURABLE OUTCOMES

For each task type, I'd define at least one leading metric (measures the system's output quality) and one lagging metric (measures actual user impact):

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q17

You're given the goal: "improve user experience using AI." How do you break this down into a concrete, scoped system with measurable outcomes?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Search improvement	retrieval precision and recall (leading), session success rate or time-to-answer (lagging)
Personalization	recommendation click-through rate (leading), session depth or return visit rate (lagging)
Automation	task completion rate (leading), time saved per user per week (lagging)
Step 4	Define what "done" looks like
Set a specific target	"reduce average time-to-answer from 3 minutes to under 45 seconds" or "increase successful search sessions from 60% to 80%." A target without a number isn't a target.
Step 5	Scope the MVP

Define the narrowest version of this that delivers measurable value. Pick one user type, one workflow, one data source. Get a number. Expand from there.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q18

You're launching an AI-powered feature in 4 weeks. How do you define success metrics before launch, and how do those metrics evolve post-launch?

A

INTERVIEW-READY ANSWER

Metrics defined after launch are justified, not measured. You need them defined before so you know what data to collect from day one and what your baseline is.

PRE-LAUNCH METRIC DEFINITION



Start by asking: what specific user behavior are you trying to change? Success metrics should connect directly to that behavior.

FOR EACH METRIC, DEFINE



What exactly you're measuring (specific, not vague)

HOW YOU'LL COLLECT THE DATA (INSTRUMENTATION NEEDED BEFORE LAUNCH)

What the baseline is (before the feature existed, what was the number?)

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q18 You're launching an AI-powered feature in 4 weeks. How do you define success metrics before launch, and how do those metrics evolve post-launch?

A INTERVIEW-READY ANSWER · CONTINUED

⚡ What success looks like at 30, 60, and 90 days post-launch

TYPICAL PRE-LAUNCH METRIC CATEGORIES FOR AI FEATURES

QUALITY METRICS (MEASURES IF THE AI IS GOOD)

POINT	WHAT IT MEANS
Task completion rate	what fraction of AI interactions result in the user getting what they needed without escalating or retrying
User correction rate	how often users modify or reject the AI's output

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q18

You're launching an AI-powered feature in 4 weeks. How do you define success metrics before launch, and how do those metrics evolve post-launch?

A

INTERVIEW-READY ANSWER · CONTINUED

ENGAGEMENT METRICS (MEASURES IF USERS USE IT)

POINT	WHAT IT MEANS
Feature adoption rate	what fraction of eligible users try the feature
Feature retention rate	what fraction of users who tried it use it again in the next 7 days

BUSINESS METRICS (MEASURES IF IT MATTERS)



Downstream conversion rate change for users of the feature vs. those who didn't use it



Support ticket deflection rate if it's a support-related feature

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q18

You're launching an AI-powered feature in 4 weeks. How do you define success metrics before launch, and how do those metrics evolve post-launch?

A

INTERVIEW-READY ANSWER · CONTINUED

HOW METRICS EVOLVE POST-LAUNCH



In the first two weeks, you're focused on catching failures: are there any catastrophic quality failures, unexpected error rates, or security issues? These are binary checks.



Weeks 3–4, you shift to optimizing leading metrics: task completion rate, correction rate, adoption.



Month 2 and beyond, you add lagging business metrics that take time to accumulate: retention, downstream conversion, revenue impact.



The quality metrics never go away. They become part of ongoing monitoring.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

This is only a selected preview.

Get the complete AI System Design Interview Kit for deeper preparation across the full curriculum.



Full question bank



Detailed answers



Follow-up questions



Real-world scenarios

↓ Check the caption to enroll